Personal research project

Diffusion models and inference-time guidance

Guiding Stable Diffusion with Differentiable Rewards

Isotropic approximation, full gradient
and comparison with LiDAR

Réda Ouzzane

11 October 2026

A study of luminance guidance in Stable Diffusion 1.5. BASE, COV, FULL and LiDAR are compared on eight held-out prompts and four seeds, after calibrating a shared guidance coefficient on five development prompts.

Abstract

A pretrained diffusion model does not necessarily produce images that satisfy a property specified after training. Inference-time guidance introduces this property through a reward. The objective studied here is to sample from a distribution that reweights the model's outputs by the exponential of that reward.

This work compares two methods that use the reward gradient. FULL differentiates the reward evaluated at the predicted clean latent through the VAE decoder and the U-Net. COV replaces the Jacobian of this prediction with an isotropic approximation motivated by a covariance identity. Both methods are compared with LiDAR, whose correction uses a bank of lookahead samples. The BASE reference retains text conditioning but applies no reward guidance.

We first validated the DDIM loop, gradients and computational-graph release on a T4. The pilots then informed the experiments: at a shared coefficient, COV produced weaker corrections than FULL; for LiDAR, better bank rewards could coincide with nearly zero guidance. These observations motivated an amplitude diagnostic and bank ablations before moving to an H100.

The H100 campaign compares all four methods on eight prompts and four seeds, with a target luminance of 0.7. Calibration on five DEV prompts selected $\beta = 115.46$. FULL improves the reward in all 32 pairs, with a mean gain of 0.03112, compared with 0.00102 for COV. LiDAR achieves a mean gain of 0.00163, with five local decreases and an amortized cost of 3.17 s per image. COV uses less memory than FULL, but their runtimes remain close: 24.34 and 25.99 s per image. FULL's reward gain comes with lower mean CLIPScore and HPSv2 than BASE. The H100 PNGs were not retrieved locally within the available time; inspecting them remains necessary to relate these scores to visual changes.

Keywords: diffusion models, inference-time guidance, differentiable reward, Tweedie's formula, DDIM, LiDAR.

Contents

1 Introduction and research question **1**

2 Mathematical framework and methods **1**
2.1 Target distribution and interpretation of the reward . . . 1
2.2 Why the guided score is difficult to compute . . . 2
2.3 BASE: DDIM with text conditioning . . . 2
2.4 FULL: differentiating the Tweedie approximation . . . 2
2.5 COV: approximating the Jacobian with an isotropic covariance . . . 3
2.6 LiDAR: a correction from lookahead samples . . . 3

3 Implementation and validation **4**
3.1 Incremental development and reusable code . . . 4
3.2 DDIM parity with Diffusers . . . 4
3.3 Checking differentiable rewards . . . 5
3.4 Identities, stability and scope of the checks . . . 5

4 Experimental protocol **5**
4.1 Shared configuration and campaign scope . . . 5
4.2 Separating development and evaluation . . . 6
4.3 Calibration and rationale for the shared coefficient . . . 6
4.4 Pairing and performance measures . . . 7
4.5 LiDAR banks, runtime and memory . . . 8
4.6 Saving, resuming and reproducibility . . . 8

5 Results and analysis **8**
5.1 Scope of the available results . . . 8
5.2 COV: response to guidance strength in the pilot . . . 9
5.3 COV and FULL: shared coefficient and amplitude diagnostic . . . 9
5.4 LiDAR: the role of the bank and weight concentration . . . 10
5.5 H100: comparison on eight prompts . . . 11
5.6 Unfavorable results and explanatory hypotheses . . . 13

6 Limitations and future work **15**
6.1 A common theoretical target without a direct fidelity measure . . . 15
6.2 Tweedie approximation and isotropic assumption . . . 15
6.3 Concentration and approximation of the LiDAR bank . . . 15
6.4 Rewards, coefficients and experimental budget . . . 15

7 Conclusion **16**

A Additional derivations **17**
A.1 Variational objective and exponentially weighted distribution . . . 17
A.2 Covariance identity . . . 17
A.3 Gradient of the LiDAR estimator . . . 17

B Additional rewards and reproduction details **17**
B.1 Functions implemented outside the main campaign . . . 17
B.2 Mapping equations to files . . . 18
B.3 H100 integration test . . . 18
B.4 Reproduction commands . . . 19

1 Introduction and research question

Diffusion models generate an image by progressively reversing a noising process. Text conditioning directs denoising towards the content described by a prompt. It does not, however, guarantee an additional criterion such as a prescribed mean luminance or spatial symmetry. Retraining the model for each criterion would be costly. Inference-time guidance seeks to change the generated outputs without modifying the model weights.

This project expresses that objective through a reward R. The target distribution favors images with higher rewards while remaining tied to the pretrained model distribution. The difficulty is to use a reward defined on a final image while sampling operates on noisy latents. Evaluating R directly on those latents is not meaningful. We must estimate what their future denoising implies for the reward.

Some guidance methods use reward values alone. This project studies what access to the gradient makes possible. We evaluate the reward at a prediction of the clean latent, then differentiate this approximation of the future value [5, 6, 7, 8]. FULL retains the U-Net derivative; COV replaces it with an isotropic approximation to reduce memory requirements. LiDAR provides a comparator without backpropagation.

The research question is: *with a fixed reward and coefficient, what does the full derivative in FULL contribute relative to the COV approximation and the LiDAR bank, in terms of reward, learned scores and cost?* The shared coefficient maintains a common theoretical target. Correction amplitudes and computational budgets remain specific to each method.

The main campaign is limited to luminance. This reward is easy to interpret, has an explicit gradient, and supports checks for sign and scaling errors. Symmetry, spatial-mask and ImageReward functions were also implemented. Their gradients were checked on one development image. They are not compared in the final campaign reported here. This narrower scope reflects the compute budget and time devoted to the initial study. The final campaign evaluates eight held-out prompts and four seeds, after fixing the shared coefficient on five development prompts. Quantitative results are analyzed separately from exploratory observations.

2 Mathematical framework and methods

2.1 Target distribution and interpretation of the reward

For a fixed prompt c, let $z_0 \in \mathbb{R}^d$ denote the clean latent and $p(z_0 \mid c)$ the reference distribution. Let D denote the VAE decoder followed by rescaling to $[0, 1]$ and pixel clipping. The reward on the latent is $r(z_0) = R(D(z_0))$. The target is

$$\pi_\beta(z_0 \mid c) = \frac{p(z_0 \mid c)\exp(\beta r(z_0))}{Z_\beta(c)}, \qquad Z_\beta(c) = \mathbb{E}_{p(\cdot|c)}[\exp(\beta r(z_0))]. \tag{1}$$

For $\beta > 0$ and a finite normalizing constant, this distribution solves

$$\max_q \left\{ \mathbb{E}_q[r(z_0)] - \frac{1}{\beta}\,\mathrm{KL}(q \,\|\, p(\cdot \mid c)) \right\}. \tag{2}$$

The first term favors reward. The second penalizes deviation from the reference. At $\beta = 0$, the target is defined by continuity and recovers p. Increasing β strengthens the reweighting. It does not guarantee better perceptual quality.

The main comparison uses the same definition of r and the same β for COV, FULL and LiDAR. The three methods therefore aim at the same *nominal* target. In the implementation, the learned score, CFG and DDIM discretization introduce further approximations. We do not assume that they sample exactly from (1).

2.2 Why the guided score is difficult to compute

The ideal noising process is

$$z_t = \alpha_t z_0 + \sigma_t \varepsilon, \qquad \varepsilon \sim \mathcal{N}(0, I), \qquad \alpha_t^2 + \sigma_t^2 = 1. \tag{3}$$

In the Stable Diffusion convention, $\alpha_t = \sqrt{\bar{\alpha}_t}$ and $\sigma_t = \sqrt{1 - \bar{\alpha}_t}$. The network predicts noise. Its output provides a score approximation: $s_\theta(z_t, t, c) = -\varepsilon_\theta(z_t, t, c)/\sigma_t$.

Noising the target distribution gives marginals $\pi_{\beta,t}$. Their score decomposes as

$$\nabla_{z_t} \log \pi_{\beta,t}(z_t \mid c) = \nabla_{z_t} \log p_t(z_t \mid c) + g_t(z_t, c), \tag{4}$$

$$g_t(z_t, c) = \nabla_{z_t} \log h_t(z_t, c), \qquad h_t = \mathbb{E}[\exp(\beta r(z_0)) \mid z_t, c]\,. \tag{5}$$

The term h_t averages an exponentiated reward over clean origins compatible with z_t. This conditional expectation is not directly accessible. Repeated denoising simulations could approximate it, but their cost would recur at every step. The methods studied here replace that operation with different approximations.

2.3 BASE: DDIM with text conditioning

The predictor combines the conditional and unconditional branches of the U-Net:

$$\varepsilon_{\text{CFG}} = \varepsilon_{\text{uncond}} + w(\varepsilon_{\text{cond}} - \varepsilon_{\text{uncond}}), \qquad w = 7.5. \tag{6}$$

BASE uses this prediction without reward correction. Here, "unguided" therefore means no guidance by R, not an absence of CFG [3].

For a transition from time t to a lower time s, deterministic DDIM applies

$$\widehat{z}_0 = \frac{z_t - \sigma_t \varepsilon}{\alpha_t}, \qquad z_s = \alpha_s \widehat{z}_0 + \sigma_s \varepsilon. \tag{7}$$

The additional-noise parameter is $\eta = 0$. This choice facilitates paired comparisons: all methods start from the same initial latent and add no random noise between steps. DDIM also supports the 50 steps chosen for this study [1]. DDPM was not added, to focus the budget on guidance approximations. Cost nevertheless depends on the number of network evaluations; a stochastic scheme is not intrinsically more expensive at the same step count.

All guided methods replace the noise prediction with

$$\varepsilon_{\text{guided}} = \varepsilon_{\text{CFG}} - \sigma_t g_t. \tag{8}$$

The same corrected prediction enters both terms of (7). Correcting only the clean prediction or only the second term would define a different update.

2.4 FULL: differentiating the Tweedie approximation

Tweedie's formula relates the score to the conditional mean of the clean latent [4]. We use the prediction

$$\mu_t(z_t) = \frac{z_t - \sigma_t \varepsilon_{\text{CFG}}(z_t, t, c)}{\alpha_t}. \tag{9}$$

If ε_{CFG} were replaced with the noise associated with the exact score of $p_t(\cdot \mid c)$, this expression would equal $\mathbb{E}[z_0 \mid z_t, c]$. Combining two exact scores through CFG does not by itself guarantee this identity. With the learned network and CFG, μ_t remains an estimate. The point approximation replaces the full conditional distribution with its mean:

$$\log h_t(z_t, c) \approx \beta r(\mu_t(z_t)). \tag{10}$$

Writing $v_t = \nabla_z r(z)|_{z=\mu_t}$, the chain rule gives

$$g_t^{\mathrm{FULL}} = \beta J_{\mu_t}^{\top} v_t, \qquad J_{\mu_t} = \frac{I - \sigma_t J_{\varepsilon_{\mathrm{CFG}}}}{\alpha_t}. \tag{11}$$

Backpropagation computes the vector–Jacobian product without constructing a dense matrix. It passes through the reward, the VAE and both CFG branches. Parameters remain frozen; the derivative is taken only with respect to the current latent.

The name FULL refers to the full derivative of (10). It does not refer to the exact gradient of (5). The expectation of an exponentiated reward generally differs from its evaluation at the mean. This difference can matter at high noise levels, where several clean images remain compatible with the same latent.

2.5 COV: approximating the Jacobian with an isotropic covariance

COV seeks to avoid backpropagation through the U-Net. For the exact conditional mean $\mu_t = \mathbb{E}[z_0 \mid z_t, c]$ under the Gaussian corruption model, the Jacobian satisfies

$$J_{\mu_t} = \frac{\alpha_t}{\sigma_t^2} \operatorname{Cov}(z_0 \mid z_t, c). \tag{12}$$

This identity concerns the true conditional mean. The prediction μ_t constructed with the learned U-Net and CFG need not satisfy it. We use the identity to motivate a heuristic approximation of its Jacobian. The covariance is replaced with that of an auxiliary isotropic model with prior variance τ^2:

$$\operatorname{Cov}(z_0 \mid z_t, c) \approx \frac{\tau^2 \sigma_t^2}{\sigma_t^2 + \alpha_t^2 \tau^2} I. \tag{13}$$

This gives

$$g_t^{\mathrm{COV}} = \beta \kappa_t v_t, \qquad \kappa_t = \frac{\alpha_t \tau^2}{\sigma_t^2 + \alpha_t^2 \tau^2}. \tag{14}$$

We set $\tau^2 = 1$, which gives $\kappa_t = \alpha_t$. This choice removes a tuning hyperparameter and provides a simple temporal rule. It does not mean that the Stable Diffusion posterior has identity covariance. Isotropy and the auxiliary variance are assumptions of the method.

The U-Net computes μ_t without retaining its graph. An independent copy of μ_t is then made differentiable to obtain v_t. The VAE and reward remain in the graph. This construction differs from simply detaching the noise prediction inside (9). The latter would produce $\beta v_t / \alpha_t$, rather than (14).

COV thus retains the gradient direction at the clean latent. It ignores the couplings and anisotropy that the full Jacobian can introduce. Its experimental interest lies in the trade-off between this loss of information and the U-Net computation it avoids.

2.6 LiDAR: a correction from lookahead samples

LiDAR uses a bank of n approximate clean latents a_i, generated independently of the evaluation seeds. Their rewards $r_i = r(a_i)$ are computed once. For a current latent z_t, the Gaussian compatibility weights are

$$L_i(z_t) = -\frac{\|z_t - \alpha_t a_i\|_2^2}{2\sigma_t^2}, \tag{15}$$

$$w_i = \operatorname{softmax}(L)_i, \qquad w_i^R = \operatorname{softmax}(L + \beta r)_i. \tag{16}$$

The derivation in Appendix A.3 then gives

$$g_t^{\mathrm{LiDAR}} = \frac{\alpha_t}{\sigma_t^2} \sum_{i=1}^{n} (w_i^R - w_i) a_i. \tag{17}$$

There is no additional factor of β in front of this sum. Its effect is already included in the reward-weighted probabilities. The method uses neither reward gradients nor backpropagation through the networks during the guided trajectory.

Our implementation uses the estimator specified above and derived in Appendix A.3. Candidates are produced by a short DDIM run. It does not reproduce all settings of the LiDAR paper, which also studies a bank solver, multipliers and specific guidance windows [8]. The reported results therefore concern this explicitly defined comparator, rather than a reproduction of the published scores.

3 Implementation and validation

3.1 Incremental development and reusable code

Development proceeded in the order BASE, rewards, COV, FULL, then LiDAR. COV was implemented before FULL to check the VAE–reward chain without retaining the U-Net graph. FULL was first tested on a single step. Complete trajectories were run after this memory-feasibility check.

The core code consists of Python functions in five scientific modules. The Colab notebooks import these functions. The Northflank runner reuses the same modules. This avoids maintaining two implementations of the equations. The reference DDIM loop and its `no_grad` decorators were preserved. Differentiable paths are separate.

The campaign uses a batch size of one. Both CFG branches are combined into a batch of two for the U-Net call. On GPU, the U-Net runs in FP16. The VAE and reward-gradient calculations use FP32. FULL retains an FP16 backward pass inside the U-Net; differentiation is therefore not entirely FP32.

After each step, the next latent is detached. FULL retains the graph for the current step, then releases it before the next. No backpropagation spans all 50 steps. Weights are frozen, with no accumulation of parameter gradients. The pilots ran without activation checkpointing or automatic transfers of modules between CPU and GPU. These optimizations were not added because no memory overflow was observed.

3.2 DDIM parity with Diffusers

The first validation compares BASE with the official pipeline using the same embeddings, initial noise tensor and timesteps. The T4 pilot covers five development prompts and seed 0. All 50 intermediate latents are compared for each generation, giving 250 states. The audited exports report exact equality. Repeating the first prompt also reproduces the same outputs.

A secondary absolute and relative tolerance of 10^{-5} was defined to detect numerical divergence. It did not replace locating the first divergent step. Visually identical images were expected: this experiment checked an implementation of the same algorithm, without a competing reward-guided method.

The original scheduler configuration is retained. It uses noise prediction with `steps_offset=1`. The remaining settings are `set_alpha_to_one=False`, `clip_sample=False` and `leading` spacing. The 50-step grid is $981, 961, \ldots, 1$. The last step uses the Diffusers fallback convention based on `alphas_cumprod[0]`. The code does not replace this coefficient with a manually chosen ideal value. The VAE scaling factor also comes from its configuration.

3.3 Checking differentiable rewards

The luminance, symmetry and spatial-mask rewards are implemented directly on image tensors. ImageReward uses the differentiable `score_gard` function from the official repository [9]. Preprocessing remains tensor-based to preserve the gradient. It includes the resizing, cropping and normalization expected by the network. Converting to a PIL image along this path would sever dependence on the latent.

Checks cover shapes, finite values and the absence of parameter gradients. The directional test covers the latent–VAE–reward path. It uses the normalized latent gradient, $d = \nabla_z r / \|\nabla_z r\|_2$; the reference derivative is therefore $\nabla_z r \cdot d = \|\nabla_z r\|_2$, with $r = R \circ D$. It is compared with a centered finite difference. All four rewards passed these checks on one final development latent.

The ImageReward finite-difference test initially failed for perturbations from 10^{-2} to 10^{-3}. The latent gradient norm was 11.23034, whereas the finite difference at 10^{-3} was 4.21918. The central score was repeatable with and without a graph. A finer grid then showed local agreement: the finite difference was 11.33703 at 10^{-4} and 11.19599 at 3×10^{-5}. The relative tolerance of 10 % was retained. The revised test requires agreement at two consecutive step sizes and local ascent. The initial failure and diagnostic are archived.

Reducing the step size restored local agreement without modifying the computed gradient. The initial diagnostic therefore used a perturbation that was too large for this test. We also compared the tensor-based ImageReward score with the PIL API score: on the test image, they were approximately -0.01472 and -0.02490, respectively. The two preprocessing procedures remain separate evaluation paths.

3.4 Identities, stability and scope of the checks

For COV, FULL and LiDAR, both $\beta = 0$ and a constant reward must recover BASE. The pilots verify equality of all 50 states, final latents and images in these cases. For LiDAR, a single-candidate bank provides a third check: $w_1 = w_1^R = 1$, so the correction is zero.

LiDAR distances are summed over latent coordinates. Replacing the sum with an average would change the kernel and hence the estimator. Distances and weights are computed in FP64 with a stable softmax. The correction is then cast to FP32, and the corrected noise to the DDIM loop's dtype. The effective sample size of the weights is recorded:

$$\mathrm{ESS}(w) = \frac{1}{\sum_i w_i^2}, \qquad 1 \le \mathrm{ESS}(w) \le n. \tag{18}$$

Both distributions, w and w^R, are monitored. An ESS close to one indicates concentration on a single candidate. On its own, it does not determine the magnitude of $w^R - w$.

The local suite documents 75 CPU tests: 44 scientific checks and 31 checks of the runner, evaluation and analysis. Orchestration tests use simulated models where no GPU is required. They check saving, resumption and aggregation, but do not validate execution of the pretrained networks on an H100.

Tests at $\beta = 0$ check limiting cases of the code; they do not guarantee that a positive coefficient improves the final reward. We therefore retain reward decreases in the experimental analysis after checking the identities and gradients.

4 Experimental protocol

4.1 Shared configuration and campaign scope

The model is `stable-diffusion-v1-5/stable-diffusion-v1-5`, loaded with Hugging Face Diffusers [2]. The weight revision is pinned. All final images have a resolution of 512×512 pixels.

Latents have shape $4 \times 64 \times 64$. All methods use 50-step DDIM, $\eta = 0$, CFG 7.5 and the same empty negative prompt.

The main reward is

$$R_{\text{lum}}(y) = -\big(\bar{\ell}(y) - \ell^\star\big)^2, \qquad \ell^\star = 0.7, \qquad \ell(y) = 0.299 y_R + 0.587 y_G + 0.114 y_B. \tag{19}$$

The mean $\bar{\ell}$ is taken over pixels. The highest possible reward is zero. Increasing this reward means bringing luminance closer to its target, not systematically brightening the image. An image initially brighter than 0.7 should be darkened.

The final H100 campaign uses the first eight prompts in the held-out evaluation set, with seeds 0, 1, 2 and 3. Each prompt–seed pair is run with BASE, COV, FULL and LiDAR. The campaign therefore contains

$$8 \times 4 \times (1 + 3) = 128 \quad \text{final images.} \tag{20}$$

The coefficient $\beta = 115.46$ is shared by the three guided methods. For COV, $\tau^2 = 1$. For LiDAR, the bank contains 16 candidates generated with five DDIM steps, using seeds distinct from the evaluation seeds. We reduced the prompt count to retain all four methods and four seeds within the available time. The initial setup covered 50 prompts and four rewards. That matrix would contain $200 + 4 \times 3 \times 200 = 2\,600$ final images, since the 200 BASE images would be reused.

The pilots ran on an NVIDIA T4 in Google Colab. Calibration and the main campaign ran on an NVIDIA H100 80 GB in Northflank, after an integration test on one DEV prompt. Reported runtimes were measured on the hardware used for each experiment.

4.2 Separating development and evaluation

The available files contain 50 development prompts and 50 held-out evaluation prompts. The two lists are disjoint and originate from `generation_prompts.txt`. Results are attributed to the lists actually used.

We selected the first eight nonempty lines of the held-out file before inspecting results. The final CSV contains exactly those prompts. Checking both lists confirms that the 50 DEV and 50 held-out prompts do not overlap. Selection by file order provides a reproducible rule; it does not guarantee that the eight prompts represent the full set of 50.

The separation concerns prompts and the use of results. Seed 0 is used during development and is also among the prescribed final seeds. This does not mean that prompt–seed pairs are reused across the two sets. LiDAR bank seeds are distinct from evaluation seeds.

4.3 Calibration and rationale for the shared coefficient

The coefficient β is a parameter of the target (1). Fixing it across methods lets us study different approximations of the same reweighting. The aim is not to find the highest possible reward for each method through independent tuning.

The first COV pilots used three states of the BASE trajectory, at indices 0, 24 and 49. The relative correction norm at $\beta = 1$ was used to set an initial grid. After implementing FULL, this calibration was repeated for both methods. The shared coefficient was defined as

$$q_m = \max_{t \in \mathcal{T}_{\text{probes}}} \frac{\|\sigma_t g_t^m(\beta = 1)\|_2}{\|\varepsilon_{\text{CFG}}\|_2}, \qquad \beta_{\text{pilot}} = \frac{0.01}{\max(q_{\text{COV}}, q_{\text{FULL}})}. \tag{21}$$

This rule gave $\beta_{\text{pilot}} \simeq 66.3703$. It controls scale only at the three probe states. It does not bound corrections along guided trajectories. LiDAR was then evaluated directly at this coefficient. Its nonlinear response to β prevents the same extrapolation from $\beta = 1$.

Further calibration was then run on the H100, with five DEV prompts, seed 0 and three shared values: $\beta \in \{23.09; 66.3703; 115.46\}$. BASE was computed once per prompt. Every

guided-method–coefficient combination was evaluated on the same five prompts. The experiment therefore contains five BASE generations and 45 guided generations, giving 50 final images. The LiDAR banks of 16 five-step candidates were shared across the three coefficients for each prompt, adding 80 short generations.

The primary criterion is paired reward gain over BASE. For method m and coefficient β, the mean DEV gain is

$$\overline{\Delta R}_m(\beta) = \frac{1}{5}\sum_{p=1}^{5}\left[R(y_{p,0}^{m,\beta}) - R(y_{p,0}^{\mathrm{BASE}})\right].$$

The mean BASE reward across these five prompts is -0.091451. Table 1 gives gains computed from the tensor rewards reported in the calibration logs.

Method	Mean gain $\beta = 23.09$	Mean gain $\beta = 66.3703$	Mean gain $\beta = 115.46$	Improved prompts $\beta = 115.46$
COV	+0.000128	+0.000920	+0.001780	5/5
FULL	+0.010628	+0.023976	+0.036983	5/5
LiDAR	+0.001404	+0.003515	+0.003867	5/5

Table 1: H100 calibration on five DEV prompts, seed 0, target luminance 0.7. Each gain is computed relative to BASE within a pair, then averaged across prompts. Coefficients are shared across methods. These observations do not come from the final evaluation set.

Among the three coefficients tested, $\beta = 115.46$ provides the highest mean gain for every guided method. At this value, all five prompts improve their reward over BASE with COV, FULL and LiDAR. This coefficient was therefore selected and fixed before examining held-out results. It is the best tested coefficient by reward on this small DEV set, not a global optimum or an optimal per-method setting. COV gains remain small at this scale, whereas FULL gains increase more markedly. LiDAR is not assumed to respond linearly to β, which enters the reward-weighted softmax. Neither CLIPScore nor HPSv2 was used to select the coefficient.

Per-method tuning would answer a different question: the trade-off achieved after optimizing each method's hyperparameters. It remains a useful extension, but does not replace comparison at a common target. Different rewards may require different coefficients because their scales differ.

4.4 Pairing and performance measures

For each prompt and seed, all methods use the same initial tensor, embeddings, model and DDIM grid. Reward differences are computed within pairs:

$$\Delta R_{p,s}^{m} = R(y_{p,s}^{m}) - R(y_{p,s}^{\mathrm{BASE}}). \tag{22}$$

A positive value indicates movement towards the target under the reward considered. The floating-point image reward is retained. Final evaluation also recomputes the reward from the PNG, so that all methods follow the same path. The two values may differ because of eight-bit quantization.

Reported statistics are mean reward, descriptive standard deviation, mean paired gain and the proportion of improved pairs. The standard deviation over 32 images uses denominator $n - 1$, with $n = 32$. It describes observed dispersion and is not a confidence interval. The four seeds for one prompt are not treated as four independent prompts. To examine differences between prompts, gains are first averaged over seeds:

$$\overline{\Delta R}_{p}^{m} = \frac{1}{4}\sum_{s=0}^{3}\Delta R_{p,s}^{m}. \tag{23}$$

The distribution of these eight means provides a complementary view of the effect. Because LiDAR shares a bank across the seeds of a prompt, this aggregation also respects a dependence introduced by the method.

CLIPScore and HPSv2 are used as metrics independent of guidance [10, 11]. CLIPScore uses CLIP ViT-B/32, the text prefix “A photo depicts” and the convention $2.5\max(\cos, 0)$. HPSv2 uses the v2.0 checkpoint and raw normalized similarity, without multiplication by 100. Versions and preprocessing are fixed in the code. Execution with pretrained weights was validated on the four PNGs of the H100 smoke test in a separate evaluation job. Test values are reported in Appendix B.3. Both metrics are also available for all 128 PNGs from the held-out campaign. They were not used to select the guidance coefficient.

CLIPScore estimates text–image alignment, whereas HPSv2 reflects learned preferences. We use them to examine the effects of luminance control on criteria that guidance does not optimize.

4.5 LiDAR banks, runtime and memory

The operational configuration contains 16 candidates per prompt, generated in five DDIM steps with seeds 10000 through 10015. The bank is built sequentially, then shared across the four final seeds. It adds $8 \times 16 = 128$ short generations to the 128 final images. Candidates are not counted as additional benchmark observations.

The main generation time includes sampling and final decoding, with CUDA synchronization. Weight loading, prompt encoding, disk writes and independent evaluation are separate from this measure. Pilots that retain CPU traces are not optimized throughput benchmarks. Peak allocated memory includes resident weights and temporary activations.

For LiDAR, bank construction and scoring must be added. With four generations sharing one bank, the amortized cost is

$$T_{\text{LiDAR,amortized}} = \frac{T_{\text{bank}} + T_{\text{scoring}} + \sum_{s=0}^{3} T_{\text{trajectory},s}}{4}. \tag{24}$$

The standalone cost of one image includes the entire bank. Both views are reported. Comparing methods at 50 steps does not equalize compute budgets: COV and FULL add decoding and backward passes, while LiDAR pays the bank cost upfront.

4.6 Saving, resuming and reproducibility

The runner is designed to save the images, latents, configurations, diagnostics and hashes needed for resumption. Before reusing a result, it checks its files and bank dependencies. Outputs are saved under `/data` on the persistent Northflank volume shared by generation and evaluation.

For the H100 campaign, we retrieved the evaluation CSV and checked all 128 observations. The 32 prompt–seed pairs are complete for all four methods; no pair is excluded. CSV persistence across jobs was checked during the integration test. The PNGs and executed-environment manifests still need to be added to the local archive to complete provenance. Appendix B records paths, code references and reproduction commands.

5 Results and analysis

5.1 Scope of the available results

The following sections distinguish the archived T4 pilots, H100 smoke test and final held-out campaign. Apart from the initial BASE validation on five prompts, the T4 pilots use the prompt “a photo of a sandwich, seed 0 and a target luminance of 0.7. Their rewards are computed on saved floating-point images. H100 tables use rewards recomputed from PNGs. The calibration gains in Section 4.3 come from five DEV prompts and tensor rewards. These sets are never pooled into a single mean.

The T4 pilots informed the next experiments: the COV/FULL diagnostic examines correction amplitude, while LiDAR ablations separate candidate count from bank generation steps. H100 smoke-test and calibration results come from the transmitted records. Final statistics are recomputed from the evaluation CSV.

5.2 COV: response to guidance strength in the pilot

The first COV pilot used a grid set from correction norms on BASE. Luminance approaches the target as β increases over the tested range (Table 2). The largest coefficient reduces absolute error by approximately 60 %. On this image, the distance to target luminance falls from 0.2678 to 0.1069.

Method	β	Luminance	Reward	Time (s)
BASE	0	0.432205	−0.071714	7.27
COV	1 181.045	0.454212	−0.060412	95.10
COV	3 543.135	0.493739	−0.042544	94.86
COV	11 810.449	0.593065	−0.011435	94.85

Table 2: First COV pilot on T4. One prompt and one seed. Runtimes include diagnostics, traces and decoding. Calibration, approximately 5.71 s, is reported separately.

All three positive-coefficient trajectories remain finite. Post-step allocated memory stays constant in the logs, with no observed accumulation. Peak allocation reaches approximately 7.92 GiB for COV, compared with 2.95 GiB for BASE. Maximum relative noise corrections are 1.02 %, 2.71 % and 5.73 %, respectively. These values differ from the levels targeted at the probes because guidance changes the trajectory itself.

Luminance control also affects local content. At the largest coefficient, the proportion of RGB components at the upper bound rises from approximately 0.44 % to 3.03 %. Saturated blacks decrease at the same time. A global saturation diagnostic must therefore distinguish both bounds. Reward alone cannot determine whether textures or contrast are better preserved.

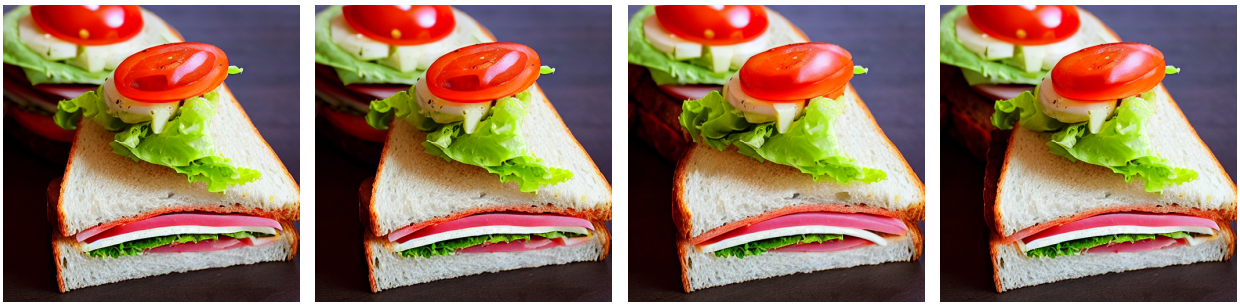
5.3 COV and FULL: shared coefficient and amplitude diagnostic

The next pilot fixed $\beta = 66.3703$ for COV and FULL. FULL brings luminance closer to the target on this image (Table 3). Its runtime and peak memory are higher. The preliminary check confirmed gradient propagation through both CFG branches. The full trajectory completed without checkpointing and without observed graph accumulation.

Method	β	Luminance	Reward	Time (s)	Peak (GiB)
BASE	0	0.432205	−0.071714	8.46	2.95
COV shared	66.3703	0.433546	−0.070998	101.45	7.92
FULL shared	66.3703	0.456291	−0.059394	120.86	9.39
COV calibrated	1 181.0449	0.454212	−0.060412	101.41	7.92

Table 3: Shared COV/FULL pilot on T4. “COV calibrated” uses a separate coefficient to obtain a comparable maximum amplitude on three BASE states. It therefore does not aim at the same nominal target as the shared-coefficient rows.

A second comparison gave COV a coefficient chosen to match the maximum correction level at the three BASE probes. Its resulting luminance, 0.454212, becomes close to FULL. This observation suggests that amplitude explains part of the gap at a shared coefficient. It does not establish that the guidance directions are equivalent. Matching amplitudes at three states does not extend to every guided step.



BASE COV shared FULL shared COV calibrated

Figure 1: Images from the COV/FULL pilot. The last column changes the COV coefficient. All four images use the same development prompt and seed.

The initial COV coefficients were therefore not transferred automatically to FULL. The shared coefficient was rescaled after measuring both gradients. This step was necessary to avoid confusing a correct method with a setting unsuited to its correction amplitude.

5.4 LiDAR: the role of the bank and weight concentration

The first LiDAR pilot used eight candidates produced with five DDIM steps. At the shared coefficient, its reward was slightly below BASE. This decrease did not invalidate the implementation: identities, weights and corrections passed the checks. It motivated a closer study of the bank.

Two ablations were run separately at $\beta = 66.3703$. The first retains the initial eight candidates and adds eight more. The second regenerates the same eight seeds with twenty DDIM steps. The time grids differ: five steps start at $t = 801$, whereas twenty steps start at $t = 951$. The second experiment therefore does not simply extend the same trajectories.

Bank	Luminance	Reward	Reduction in $\lvert\bar{\ell} - 0.7\rvert$	Total cost (s)
8×5	0.432017	−0.0718150	−0.070 %	16.97
16×5	0.434458	−0.0705127	0.841 %	30.23
8×20	0.433736	−0.0708963	0.572 %	40.19

Table 4: LiDAR ablations on the same prompt and seed. Total cost includes the bank, its decoding/scoring and one final generation. Error reduction is measured relative to BASE. Reference values come from the historical pilot.

The 16×5 bank provides the best observed trade-off among these three configurations. Its total cost is 30.23 s, even though the additional computation actually launched cost only 21.13 s because eight candidates were reused. Omitting their historical cost would artificially favor this ablation. Peak allocations in the new trials are close to 2.95 GiB. Measurements from different sessions do not constitute a repeated throughput benchmark.

Diagnostic over 50 steps	8×5	16×5	8×20
Steps with $\mathrm{ESS}(w^R) < 1.01$	42	26	49
Exactly zero noise correction	10	18	41
Maximum relative correction	2.93 %	11.99 %	0.72 %

Table 5: LiDAR diagnostics. Relative correction compares the change in predicted noise with the CFG noise norm. It does not directly measure relative latent displacement.

Adding candidates reduces weight concentration along part of the trajectory. It does not eliminate it. The number of exactly zero corrections even increases between 8×5 and 16×5. ESS alone therefore does not describe all guidance activity.

The 8×20 result is informative. Its bank contains higher-reward candidates, but guidance becomes almost zero after the first steps. The reason is visible in (17): when w and w^R concentrate on the same candidate, their difference approaches zero. The highest-reward bank image does not necessarily receive a large weight, because compatibility with z_t also matters. Improving bank rewards therefore does not guarantee a more useful correction.

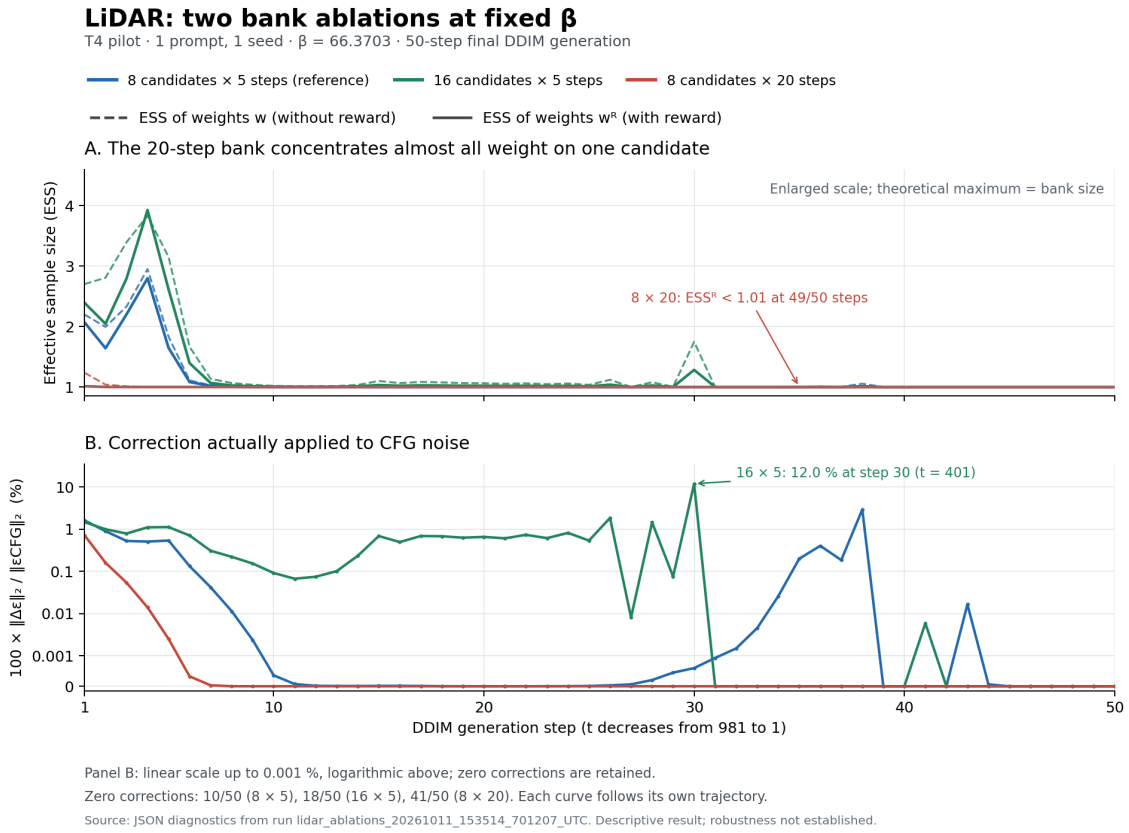


Figure 2: Weights and corrections along the three LiDAR trajectories. Each curve follows its own guided trajectory. The lower panel retains zero corrections and uses a linear scale near zero, then a logarithmic scale. Source: archived T4 pilot and ablation diagnostics.

The 11.99 % correction peak for 16×5 occurs at time $t = 401$. Two candidates compete at this point. The reward changes their relative weights enough to produce a localized correction. This event does not establish instability, but shows that calibration on a few states does not bound every step.

We selected the 16×5 bank for the H100 campaign: on the pilot, it improves reward more than 8×20 at a lower total cost.

5.5 H100: comparison on eight prompts

An H100 test on one DEV prompt produced all four images and validated independent computation of reward, CLIPScore and HPSv2 on the saved PNGs (Appendix B.3). We then ran the held-out campaign at $\beta = 115.46$: eight prompts, four seeds and four methods, giving 128 images. All 32 prompt–seed pairs are complete. The following means exclude the initial test and DEV calibration.

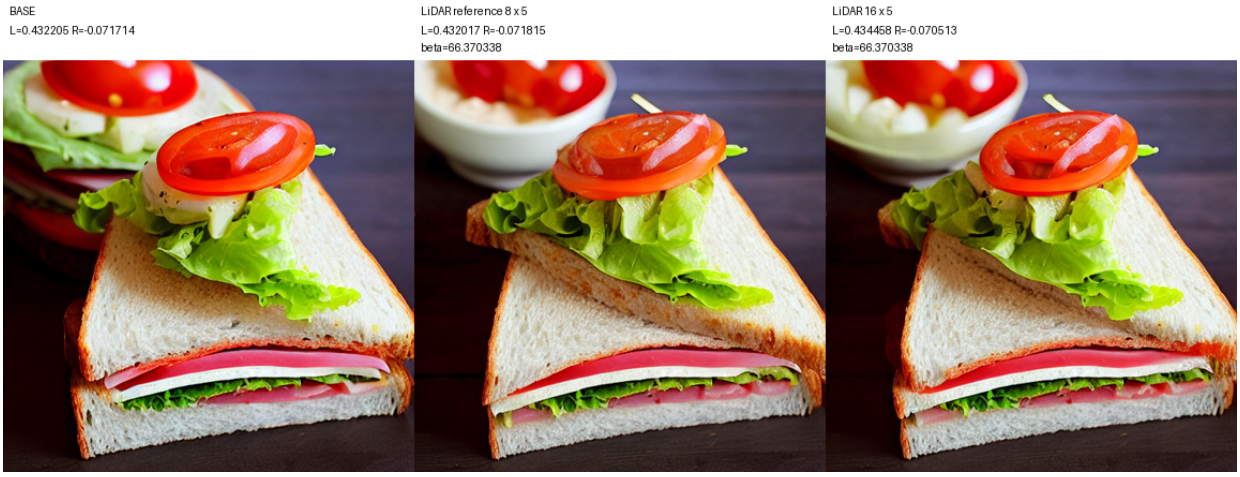


Figure 3: Left to right: BASE, LiDAR 8×5 and LiDAR 16×5, at the shared coefficient. Composition changes are visible despite the small luminance gain. These images come from the T4 pilot.

The luminance reward reaches its maximum at zero. Table 6 shows that FULL improves this reward in all 32 paired comparisons. Its mean gain is 0.03112. COV also improves every pair, but with a mean gain of 0.00102. LiDAR achieves a mean gain of 0.00163 and improves 27 of 32 pairs. Its positive mean therefore masks five local decreases.

Method	Images	PNG reward mean ± standard deviation	Mean gain vs BASE	Improved pairs
BASE	32	−0.05856 ± 0.02389	reference	—
COV	32	−0.05754 ± 0.02329	+0.00102	32/32
FULL	32	−0.02744 ± 0.01131	+0.03112	32/32
LiDAR	32	−0.05693 ± 0.02311	+0.00163	27/32

Table 6: Main H100 campaign: eight held-out prompts and four seeds, giving 32 images per method. Gains are paired and rewards are recomputed from PNGs. Standard deviation uses denominator $n - 1$; it describes the 32 observations and does not quantify uncertainty across 32 independent prompts.

FULL improves luminance reward, but its mean CLIPScore and HPSv2 decrease by 0.00906 and 0.00218, respectively, relative to BASE (Table 7). For each metric, the mean gain over four seeds is negative for five prompts and positive for the other three. COV and LiDAR remain close to BASE, with small positive mean differences.

Method	Mean CLIPScore	Δ CLIPScore vs BASE	Mean HPSv2	Δ HPSv2 vs BASE
BASE	0.77716	reference	0.28175	reference
COV	0.77801	+0.00086	0.28185	+0.00009
FULL	0.76810	−0.00906	0.27957	−0.00218
LiDAR	0.77953	+0.00238	0.28283	+0.00108

Table 7: Independent metrics on the same 128 PNGs. Differences are computed before rounding. Neither CLIPScore nor HPSv2 is used in luminance guidance or in selecting β.

Mean trajectory cost rises from 1.91 s for BASE to 24.34 s for COV and 25.99 s for FULL (Table 8). Here, the isotropic approximation reduces peak allocation from 9.39 to 7.92 GiB relative to FULL, but leaves runtime close. COV still requires repeated decoding and backward passes.

For LiDAR, the eight distinct banks cost 36.82 s in total, including generation and scoring. Metadata repeated across four seeds are counted once per bank. Mean preparation cost is therefore 4.60 s per prompt, or 1.15 s per image after amortization. Added to the 2.02 s trajectory cost, this gives 3.17 s per image. A standalone image bearing the entire preparation cost would cost 6.62 s on average in these measurements. The 2.02 s trajectory alone is therefore insufficient for comparing LiDAR cost.

Method	Trajectory (s/image)	With amortized bank (s/image)	Maximum allocated peak (GiB)
BASE	1.91	1.91	2.98
COV	24.34	24.34	7.92
FULL	25.99	25.99	9.39
LiDAR	2.02	3.17	2.98

Table 8: Costs measured on H100. Mean runtimes include sampling and final decoding; they exclude loading, text encoding, writes and independent evaluation. LiDAR bank cost is spread across four seeds. Memory is the maximum PyTorch allocation peak across 32 trajectories, not total process memory. The CSV does not report peak memory for bank construction.

Per-prompt analysis qualifies the global mean (Table 9). FULL and COV have a positive mean gain for each of the eight prompts. For LiDAR, seven means are positive. The prompt describing a yellow keyboard and a black sink is the exception, with a mean gain of −0.00044. FULL's highest mean gain, 0.05579, occurs for the fork and knife.

Prompt (index)	COV gain	FULL gain	LiDAR gain
0 — a photo of a teddy bear	+0.00090	+0.02749	+0.00104
1 — a photo of a computer keyboard and a microwave	+0.00051	+0.02160	+0.00388
2 — a photo of two toothbrushs	+0.00075	+0.02934	+0.00278
3 — a photo of a purple backpack	+0.00097	+0.03060	+0.00079
4 — a photo of a zebra below a broccoli	+0.00122	+0.02861	+0.00245
5 — a photo of a yellow computer keyboard and a black sink	+0.00076	+0.02382	−0.00044
6 — a photo of a potted plant	+0.00137	+0.03174	+0.00022
7 — a photo of a fork and a knife	+0.00170	+0.05579	+0.00228

Table 9: PNG reward gain over BASE, averaged over the four seeds of each prompt. Labels reproduce the metadata, including the spelling "toothbrushs".

H100 metrics were computed remotely on the saved PNGs. Their local transfer did not complete within the available time, so we did not examine paired image panels from this campaign. The T4 panels shown above were inspected. FULL's lower learned scores must be compared with the images before being interpreted as perceptual degradation.

5.6 Unfavorable results and explanatory hypotheses

Three findings call for explanation: the five LiDAR reward decreases, COV's small gain despite its cost, and the mean decrease in learned scores with FULL.

LiDAR: five local decreases

LiDAR improves 27 of 32 pairs, with a mean gain of 0.00163. The prompt "a photo of a yellow computer keyboard and a black sink" retains a negative mean gain after aggregation over four seeds (−0.00044). The bank is a first possible explanation.

The correction depends on the difference between w^R and w. When both distributions concentrate their mass on the same candidate, this difference becomes nearly zero. In the T4 pilot, the 8×20 bank contained higher-reward candidates, yet 41 of 50 noise corrections were exactly zero. This observation explains why improving candidates alone does not make guidance active. It does not yet connect that mechanism to the five H100 decreases.

For these five pairs, we would need to examine the trajectories of ESS(w) and ESS(w^R), dominant candidates and correction norms. These diagnostics are absent from the retrieved CSV. A DEV ablation would then vary candidate count and bank generation steps separately to measure their effects.

COV: a small gain despite its cost

COV improves every pair, but its mean gain of 0.00102 remains small compared with FULL's 0.03112. Savings are mainly in memory: peak allocation falls from 9.39 to 7.92 GiB, while runtime falls only from 25.99 to 24.34 s per image. Avoiding U-Net backpropagation does not remove repeated VAE decoding and reward gradients.

Two explanations for the small gain remain to be distinguished. The isotropic approximation may orient the correction less effectively than the full Jacobian. It may also produce too small an amplitude at the shared coefficient, since $\tau^2 = 1$ gives $\kappa_t = \alpha_t$. In the T4 pilot, increasing the COV coefficient to approximate FULL's amplitude on three states brought their rewards closer. This supports the amplitude hypothesis for that case; directions were not compared.

The next test would measure norms and cosines between corrections on the same DEV latents, then compare generations with controlled amplitudes. Separate β tuning for each method would complement this study, with a target different from the current comparison. Profiling the U-Net, VAE and backward passes would also identify the cost that COV actually avoids.

FULL: luminance and learned scores

FULL improves luminance reward in all 32 pairs. Other metrics respond less uniformly: CLIPScore decreases in 21 pairs, while HPSv2 increases in 17 and decreases in 15. The case "a photo of a zebra below a broccoli", seed 0, produces a CLIPScore decrease of 0.17852 and contributes strongly to the negative mean.

The optimized reward concerns global luminance only. It does not explicitly require preservation of objects, their relationships or contrast. Our hypothesis is that some luminance corrections also change features used by the learned metrics. The coefficient was selected on DEV using reward, without optimizing CLIPScore or HPSv2.

To test this hypothesis, we would inspect the four images in pairs with the largest CLIPScore decreases, looking for changes in content and contrast. Varying β on DEV would then measure the joint response of luminance and both scores. These experiments would help distinguish changes in visual content from sensitivity of the metrics to appearance.

ImageReward: failure of the initial diagnostic

The first ImageReward finite-difference test failed with overly large step sizes. Smaller perturbations restored local agreement without changing the computed gradient (Section 3.3). We therefore corrected the diagnostic, not the method. This episode led us to retain a step-size grid and check local convergence before interpreting a discrepancy with autograd.

6 Limitations and future work

6.1 A common theoretical target without a direct fidelity measure

The shared coefficient makes the protocol consistent with approaching the same distribution. It does not measure that proximity. One method may increase reward by concentrating its outputs excessively. Another may produce a lower reward while preserving more variability. The computed metrics cannot directly distinguish these behaviors through a divergence from π_β.

A useful extension would be a low-dimensional problem with an analytically known target distribution. It would allow comparison of sample means, covariances and distributional discrepancies. This test would complement the current algebraic identities. It would not replace the Stable Diffusion experiment, but would help separate approximation error from learned-network error.

6.2 Tweedie approximation and isotropic assumption

FULL replaces the conditional expectation with a point evaluation. It therefore ignores the spread of clean images compatible with a noisy latent. Nonlinearity in the VAE and reward may make this substitution inaccurate. Even an exactly computed gradient of this approximate function can point towards a region that is not useful for the final reward.

COV adds an isotropy assumption. It ignores directions of high uncertainty and dependencies between latent coordinates. The parameter $\tau^2 = 1$ was not estimated from the posterior. Ablations on this scale or a diagonal approximation would help study this limitation. These variants would change the Jacobian approximation defined in Section 2.5.

Mixed precision is another limitation. Small corrections may be attenuated when cast back to FP16. Diagnostics measure the difference between the intended correction and the one actually applied. A more expensive FP32 study could quantify this effect at a few shared states.

6.3 Concentration and approximation of the LiDAR bank

A small bank imperfectly represents a high-dimensional latent distribution. Gaussian distances may concentrate almost all weight on one candidate. Short generations introduce bias because five DDIM steps do not yield exact samples from the reference law. Sharing a bank across seeds reduces cost but introduces dependence between their results.

The T4 ablations motivate separating bank size from lookahead generation accuracy in future trials. The diagnostics proposed in Section 5.6 would connect these parameters to guidance activity. Modifying the compatibility kernel would change the estimator and require a separate derivation.

6.4 Rewards, coefficients and experimental budget

The final campaign concerns one global property: luminance. Symmetry, spatial-mask and ImageReward gradients are implemented and locally validated; their guided generations remain to be compared. We would start with a spatial reward to test whether the differences between FULL, COV and LiDAR persist when the criterion depends on pixel arrangement.

The shared coefficient 115.46 is the best of three tested on five DEV prompts and one seed. A wider grid would measure sensitivity to this target. Tuning a separate coefficient for each method would answer a different question: performance after optimizing its hyperparameters.

The eight prompts limit the diversity of evaluated content. The four seeds measure variability conditional on a prompt; they do not create 32 independent prompts. In addition, the quadratic reward in the CSV cannot determine whether an image is brighter or darker than 0.7. Luminance values or PNGs would need to be recovered to isolate cases where guidance must darken the image.

Retrieving the PNGs is also the next step for examining content preservation and visual diversity. Published costs include the amortized LiDAR bank but not all tuning, transfers and evaluation. Peak memory during bank construction still needs to be extracted from the logs to complete the hardware comparison.

7 Conclusion

We compared two ways of using reward gradients: FULL retains the denoiser Jacobian, while COV replaces it with an isotropic approximation motivated by the covariance identity. LiDAR estimates the correction from an independent bank. The pilots highlighted two practical choices: correction amplitude and how LiDAR weights distribute mass across candidates.

On eight prompts and four seeds, at $\beta = 115.46$, FULL achieves the largest mean reward gain (+0.03112) and improves all 32 pairs. It costs 25.99 s per image and has lower mean CLIPScore and HPSv2 than BASE. COV reduces peak memory, but its gain of 0.00102 remains small at a runtime close to FULL. LiDAR is the least expensive tested guidance method after amortizing its bank (3.17 s per image); its mean gain of 0.00163 includes five local decreases.

These experiments identify two priorities: separate amplitude and direction effects for COV, then relate LiDAR failures to bank geometry and weights. Inspecting H100 images will complement the analysis of FULL's learned scores. Evaluating other rewards will then test whether these findings extend beyond luminance control.

A Additional derivations

A.1 Variational objective and exponentially weighted distribution

For any distribution q absolutely continuous with respect to p, equation (1) implies

$$\mathrm{KL}(q\|\pi_\beta) = \mathrm{KL}(q\|p) - \beta\mathbb{E}_q[r] + \log Z_\beta. \tag{25}$$

The objective (2) therefore equals $\beta^{-1}(\log Z_\beta - \mathrm{KL}(q\|\pi_\beta))$. It is maximized at $q = \pi_\beta$. This relation explains the role of the shared coefficient. It does not make the divergence of generated images computable when their density is unknown.

A.2 Covariance identity

In the Gaussian model, write $\mu_t = \mathbb{E}[z_0 \mid z_t]$. The derivative of the conditional log-density is

$$\nabla_{z_t} \log p(z_0 \mid z_t) = \frac{\alpha_t}{\sigma_t^2}(z_0 - \mu_t). \tag{26}$$

Differentiating the mean under the integral, when this interchange is justified, gives

$$J_{\mu_t} = \frac{\alpha_t}{\sigma_t^2}\mathbb{E}[z_0(z_0 - \mu_t)^\mathsf{T} \mid z_t] = \frac{\alpha_t}{\sigma_t^2}\,\mathrm{Cov}(z_0 \mid z_t). \tag{27}$$

For a Gaussian prior with covariance $\tau^2 I$, the conditional covariance is $(\tau^{-2} + \alpha_t^2/\sigma_t^2)^{-1} I$. Substitution gives (14). The auxiliary model is used only to approximate the Jacobian; it does not replace the learned denoiser when computing μ_t.

A.3 Gradient of the LiDAR estimator

The estimator can be written as a ratio of two sums:

$$\log \widehat{h}_t = \log \sum_i \exp(L_i + \beta r_i) - \log \sum_i \exp(L_i). \tag{28}$$

Because the candidates are fixed, $\nabla_{z_t} L_i = -(z_t - \alpha_t a_i)/\sigma_t^2$. Differentiating each log-sum-exp produces its softmax weights. The terms in z_t cancel because both weight distributions sum to one. This leaves exactly (17). The derivation explains the zero-correction cases: $\beta = 0$, constant reward and a singleton bank.

B Additional rewards and reproduction details

B.1 Functions implemented outside the main campaign

In addition to (19), the following functions were implemented:

$$R_{\mathrm{sym}}(y) = -\frac{1}{3HW}\|y - \mathrm{flip}(y)\|_2^2, \tag{29}$$

$$R_{\mathrm{mask}}(y) = -\frac{1}{HW}\|\ell(y) - m\|_2^2. \tag{30}$$

The tested mask is a bright central disk on a dark background, with radius one quarter of the shorter side. ImageReward provides the fourth function, conditioned on the prompt. These paths were checked on one final development latent. No multi-prompt guidance campaign using these three rewards is claimed here.

B.2 Mapping equations to files

File	Role in the experiment
`baseline.py`	Model, CFG embeddings, initial noise, explicit DDIM and cost measurement.
`rewards.py`	Differentiable decoding, rewards and gradient checks.
`guidance_cov.py`	Isotropic approximation in equation (14).
`guidance_full.py`	Vector–Jacobian product in equation (11).
`guidance_lidar.py`	Weights, ESS and correction in equation (17).
`run_experiment.py`	Task execution, banks, saving and resumption.
`evaluate.py`	PNG rewards and independent metrics.
`analyze_results.py`	Descriptive tables, paired gains and cost accounting.

Direct dependencies include PyTorch 2.8.0, Diffusers 0.35.1, Transformers 4.51.3, Accelerate 1.10.1 and NumPy 2.1.3. Transitive versions should be retained in the actual campaign manifest. The SD1.5 revision used is

`451f4fe16113bff5a5d2269ed5ad43b0592e9a14`.

The code for this personal project is collected at

`https://github.com/redaouzz/diffusion-reward-guidance`

The eight-prompt configuration retains its historical filename. This edition gathers existing sources and results; it does not correspond to a new campaign. The SHAs and digests actually executed on H100 still need to be confirmed from remote manifests.

The `validation/` directory retains T4 export audits and CPU checks. The final CSV is at `data/benchmark_evaluation.csv`. The file `validation/release_manifest.json` identifies the materials in this edition and preservation checks. Paths and project names in some archived metadata have been normalized; measurements and historical hashes are retained. The report's three figures come from archived T4 artifacts. H100 PNG hashes have not been verified in this archive.

B.3 H100 integration test

Before calibration, the H100 test ran all four methods on the DEV prompt "a photo of a sandwich", seed 0, with $\beta = 66.3703$ for guided methods. A separate evaluation job reloaded the four PNGs and computed reward, CLIPScore and HPSv2. Generation and evaluation completed without error. The CSV was found in a subsequent container, confirming persistence across jobs.

Method	β	PNG reward ↑	CLIPScore ↑	HPSv2 ↑	Time (s)	Peak (GiB)
BASE	0	−0.071536	0.7357	0.2908	2.57	2.95
COV	66.3703	−0.070885	0.7348	0.2913	24.50	7.92
FULL	66.3703	−0.059193	0.7406	0.2920	26.29	9.39
LiDAR	66.3703	−0.070563	0.7246	0.2905	2.13*	2.98

Table 10: H100 smoke test on one DEV prompt and one seed. Values recorded in `/data/results/h100_smoke/evaluation.csv`. Rewards are recomputed from PNGs. Runtimes cover sampling and final-image decoding. *For LiDAR, the bank (2.96 s) and its scoring (1.64 s) are separate: adding them gives approximately 6.73 s from the rounded values. Peak memory is converted from MiB to GiB.

FULL achieves the highest reward here, with CLIPScore and HPSv2 above BASE. This observation on one development image is kept separate from held-out means. In the final CSV, the maximum absolute difference between PNG and tensor reward is 2.5444×10^{-6}.

B.4 Reproduction commands

Enter each command below on a single line. They retain the paths reported for the Northflank jobs. The initial test uses `configs/t4_smoke.json`, whose name does not require a T4 GPU:

```
python run_experiment.py --config configs/t4_smoke.json
  --output /data/results/h100_smoke --device cuda
```

DEV calibration used five prompts, the script's default seed 0 and three shared coefficients:

```
python calibrate_beta.py
  --prompts data/prompts_50_dev.txt
  --exclude-prompts data/prompts_50_heldout.txt
  --limit 5 --betas 23.09 66.37033847314085 115.46
  --output /data/results/beta_calibration_5p3b --device cuda
```

Independent evaluation of the initial test reloaded PNGs from the volume:

```
python evaluate.py --input /data/results/h100_smoke
  --metrics reward clip hps --device cuda
  --cache-dir /data/cache
```

The main configuration contains eight prompts, four seeds, luminance as the sole reward and the fixed shared coefficient. It replaces use of the historical 50-prompt, four-reward template:

```
python run_experiment.py
  --config configs/benchmark_luminance_10p.json
  --output /data/results/benchmark_luminance_8p --device cuda
```

Independent evaluation of the main campaign uses:

```
python evaluate.py --input /data/results/benchmark_luminance_8p
  --metrics reward clip hps --device cuda
  --cache-dir /data/cache
```

The shared 60 GB volume is mounted at `/data` in `ReadWriteMany` mode. Metric weights are downloaded at runtime to the persistent cache. The configuration `benchmark_luminance_10p.json` retains a historical name; its limit is eight prompts. The final CSV was transferred from the Northflank shell. It establishes that all 128 expected evaluated results are present, but does not establish total wall-clock duration, the number of resumptions or the job bill.

References

[1] J. Song, C. Meng and S. Ermon. *Denoising Diffusion Implicit Models*. ICLR, 2021. https://arxiv.org/abs/2010.02502.

[2] R. Rombach et al. *High-Resolution Image Synthesis with Latent Diffusion Models*. CVPR, 2022. https://arxiv.org/abs/2112.10752.

[3] J. Ho and T. Salimans. *Classifier-Free Diffusion Guidance*. 2022. https://arxiv.org/abs/2207.12598.

[4] B. Efron. *Tweedie's Formula and Selection Bias*. Journal of the American Statistical Association, 106(496), 2011, p. 1602–1614. https://doi.org/10.1198/jasa.2011.tm11181.

[5] M. Uehara et al. *Inference-Time Alignment in Diffusion Models with Reward-Guided Generation: Tutorial and Review*. 2025. https://arxiv.org/abs/2501.09685.

[6] H. Chung et al. *Diffusion Posterior Sampling for General Noisy Inverse Problems*. ICLR, 2023. https://arxiv.org/abs/2209.14687.

[7] A. Bansal et al. *Universal Guidance for Diffusion Models*. CVPR Workshops, 2023. https://arxiv.org/abs/2302.07121.

[8] Y. Kim et al. *Lookahead Sample Reward Guidance for Test-Time Scaling of Diffusion Models*. ICML, 2026. arXiv version v2. https://arxiv.org/abs/2602.03211v2.

[9] J. Xu et al. *ImageReward: Learning and Evaluating Human Preferences for Text-to-Image Generation*. NeurIPS, 2023. https://arxiv.org/abs/2304.05977.

[10] J. Hessel et al. *CLIPScore: A Reference-free Evaluation Metric for Image Captioning*. EMNLP, 2021. https://arxiv.org/abs/2104.08718.

[11] X. Wu et al. *Human Preference Score v2: A Solid Benchmark for Evaluating Human Preferences of Text-to-Image Synthesis*. 2023. https://arxiv.org/abs/2306.09341.